

Расписание станков

Это по большей части задача на реализацию, требующая при этом воспользоваться идеей ленивых вычислений. Разберем ее по частям.

Начнем с того, что, собственно, требуется хранить:

1. Для каждой задачи необходимо знать время начала и завершения ее обработки. Поскольку эти значения могут быть выставлены как последовательно при обработке, так и в любой момент времени при отмене, имеет смысл просто иметь доступ к задаче по ее номеру за $\mathcal{O}(1)$: заведем словарь (unordered map, hash map) `jobs`, сопоставляющий номера задач и сами задачи.
2. Также для задачи полезно понимать, срочная ли она (`job.is_urgent`), и какой прогресс по ней уже выполнен (`job.progress`). Ну и саму длительность задачи `job.duration` тоже запоним отдельным полем. Определим `job.remaining()` как 0, если она отменена, и как разницу между длительностью и прогрессом иначе.
3. Для каждого станка в любой момент времени может понадобиться узнать время его остановки. Мы будем хранить это отдельным полем `machine.finish` для каждого станка, так как это самый эффективный способ быстро пересчитывать это значение при добавлении или отмене задач.
4. Разумеется, для каждого станка надо хранить указанную в условии очередь задач `machine.queue`.
5. И для обновления `machine.finish` надо хранить словарь, сопоставляющий задачи станкам, на которых они обрабатываются.

Однако этого недостаточно. В этой задаче необходимы ленивые вычисления по одной причине: отмененная задача может находиться в середине очереди, из-за чего ее явное удаление оттуда может быть достаточно долгой операцией. Можно было бы в качестве альтернативы хранить упорядоченное множество вместо очереди, однако это замедлит и усложнит решение.

Идея ленивых вычислений в следующем: вместо реального удаления задач будем просто пометить задачу удаленной (`job.is_cancelled`), а удалим ее из очереди, когда очередь до нее дойдет. Вторая идея — обрабатывать задачи не в режиме реального времени, а по необходимости. Действительно, станки независимы, поэтому можно проделать всю работу станка между предыдущим и текущим моментом времени только тогда, когда есть необходимость, например, понять где сейчас начало очереди. Поэтому будем дополнительно для каждого станка хранить последний момент времени, на котором мы «остановились» `machine.t`.

Тогда заведем следующие методы:

```
func (machine) work_until(time):
    while machine.t < time and len(machine.queue) > 0:
        job = machine.queue[0]
        if job.is_cancelled:
            machine.queue.popleft()
            continue

        machine.process_job(job, time)
        if job.remaining() == 0:
            machine.queue.popleft()

    machine.t = time
```

Иными словами, при необходимости в момент времени t узнать актуальное состояние станка, выполним всю его работу до этого момента: будем увеличивать прогресс по работам по очереди, пока не дойдем до текущего момента. При этом `process_job` выглядит следующим образом:

```

func (machine) process_job(job, time):
    rem = min(time - machine.t, job.remaining())
    if job.start is None:
        job.start = machine.t

    job.progress += rem
    machine.t += rem
    if job.remaining():
        job.finish = machine.t

```

Здесь мы увеличиваем прогресс по задаче на минимум из того, сколько осталось времени до ее завершения и до достижения момента `time`. Не забываем при этом выставить начало и конец обработки задачи в соответствии с определениями из условия.

Остается только аккуратно обработать все запросы. Перед выполнением запроса к станку вызываем для него `work_until`, проверяем, что запрос не игнорируется, добавляем задачу в очередь и увеличиваем `machine.finish` на ее длительность. При отмене задачи просто помечаем задачу отмененной, выставляем ей конец обработки и, возможно, начало, после чего уменьшаем `machine.finish` для соответствующего станка на `job.remaining()`.

В итоге получаем решение, работающее за время $\mathcal{O}(n + q)$, хотя в некоторых тестах и решение за $\mathcal{O}(nq)$ могло пройти по времени.

В этой задаче было два варианта, и они отличались определением времени начала, форматами ввода и вывода и небольшой заменой одного из запросов. Все детали реализации во втором варианте остаются теми же, только, из основных изменений, время начала обработки задачи следует выставлять в другом месте: при добавлении задачи, если очередь пустая или задача срочная, а также при завершении обработки предыдущей в очереди задачи. Для этого удобно в `work_until` заменить `while machine.t < time` на `while machine.t <= time`.

Бинарная Сила

В этой задаче можно было попробовать использовать жадные идеи — стараться сначала выполнять более дорогие квесты, но они не дали бы оптимальный ответ в большей части случаев. Так что будем оптимизировать полный перебор всех вариантов с помощью динамического программирования.

Динамическое программирование позволяет нам заметить, что если бы мы делали полный перебор, то при выборе навыка в очередной вершине нам были бы важны только значения навыков в ее детях, но не ниже. Соответственно, достаточно не перебирать все возможные распределения навыков по дереву, а только для каждой пары из вершины и ее возможного навыка определить оптимальный ответ на ее поддереве. Введем $\text{dp}[v][i]$ — оптимальный ответ на поддереве v , если в ней стоит навык i .

Теперь надо научиться пересчитывать эти значения. Порядок пересчета понятен — просто считаем снизу вверх по дереву, тогда при вычислении $\text{dp}[v]$ значения dp для ее детей уже посчитаны. Теперь заметим, что случаи, когда i находится в левом поддереве v , и в правом, симметричны, поэтому далее мы разберем только первый случай, а второй будет аналогичен.

Для удобства реализации пронумеруем все вершины сверху вниз и слева направо, начиная с нуля. Тогда несложно заметить, что дети вершины v будут иметь номера $2v + 1$ и $2v + 2$. При этом номер слоя вершины v будет равен $\lfloor \log_2(v + 1) \rfloor$, что мы обозначим за $\text{layer}(v)$, а номера листьев в ее поддереве (в нумерации с нуля) будут лежать в интервале от $\min(v) = (v + 1) \cdot 2^{n - \text{layer}(v)} - 2^n$ до $\max(v) = \min(v) + 2^{n - \text{layer}(v)}$ не включительно. Скажем, что $\text{dp}[v][i]$ отвечает за лист номер $\min(v) + i$.

При этом также сразу оценим, что сумма количества листьев по всем поддеревьям дерева имеет размер порядка $2^n \cdot n$, так как каждый лист входит в n поддеревьев. Поэтому если мы будем пересчитывать $\text{dp}[v][i]$ за $O(1)$, полная асимптотика времени решения будет равна $O(2^n \cdot n)$.

Пусть $i \in [\min(v), \frac{\min(v) + \max(v)}{2})$, то есть находится среди левой половины листьев в поддереве v . Тогда $\text{dp}[v][i]$ подразумевает, что в вершине $2v + 1$ (в левом ребенке) также был выбран навык номер i . При этом в правом ребенке может быть выбран любой навык, поэтому

$$\text{dp}[v][i] = \text{dp}[2v + 1][i] + \max_{j \in [\min(2v+2), \max(2v+2))} \text{dp}[2v + 2][j] + \text{REWARD}.$$

Здесь **REWARD** означает количество монет, которое мы получим за размещение i -го навыка в v -й вершине. Давайте считать, что все квесты типа «less» изначально выполнены (то есть в базе динамики положим значения, равные сумме монет за все квесты типа «less» с уровнем больше 0), тогда за такое действие мы получим:

- минус число монет за квесты «less» для i -го навыка и уровня, соответствующего v (действительно, до этого ограничение квеста выполнялось, а теперь перестало);
- монеты за квесты «exact» для i -го навыка и уровня, соответствующего v ;
- монеты за квесты «more» с тем же уровнем навыка по аналогичным причинам.

Чтобы быстро считать это число монет, заранее сгруппируем все квесты по навыкам, а в каждой группе — по уровню. И для каждого уровня сложим монеты из квестов «exact» и «more» и вычтем монеты за квесты «less». Это и будет значение **REWARD**, используемое выше.

Остается только заметить, что $\max_j \text{dp}[v][j]$ тоже можно поддерживать во время пересчета dp в отдельном массиве max_dp , добавив $O(1)$ времени на каждый пересчет. Тогда в сумме весь пересчет отработает за время $O(2^n \cdot n)$, а ответ в конце будет находиться в $\text{max_dp}[0]$.